Assembly Language Project Report

Parking Management System (EMU8086)

Title Page

Submitted by: Muhammad Haseeb
Platform: EMU8086 / DOS Environment
Language: 8086 Assembly Language

Course: Computer Organization & Assembly Language

Githutb Link: https://github.com/MuhammadHaseeb54/Parking_System_Assembly_Project

Abstract

The Parking System is developed using 8086 Assembly Language in the EMU8086 DOS environment. The system manages vehicle parking records for rikshaws, cars, and buses. It allows the user to add vehicles, display parking records, calculate parking amounts, delete the last parking entry using stack logic, and handle invalid inputs.

The project demonstrates the practical implementation of Assembly Language concepts such as interrupts, loops, procedures, stack operations, variables, and conditional branching.

Introduction

8086 Assembly Language is a low-level programming language used to directly interact with hardware and memory. It provides complete control over CPU registers and system resources.

The Intel 8086 microprocessor uses registers, memory addressing, interrupts, and instructions to perform operations. Assembly language programs are efficient and fast because they execute close to hardware level.

Important concepts used in this project:

- Registers (AX, BX, DX, SI)
- Procedures
- Conditional jumps
- DOS interrupts
- Stack operations
- Loops and counters

Project Overview

This project is a console-based Parking System developed in Assembly Language. The system manages parking records for:

- Rikshaws
- Cars
- Buses

The system stores vehicle records, calculates parking charges, displays reports, and removes the last inserted record using stack implementation.

Objectives

- To understand 8086 Assembly programming
- To implement parking system logic
- To use DOS interrupts for input/output
- To demonstrate stack operations
- To calculate total parking amount
- To handle invalid user inputs

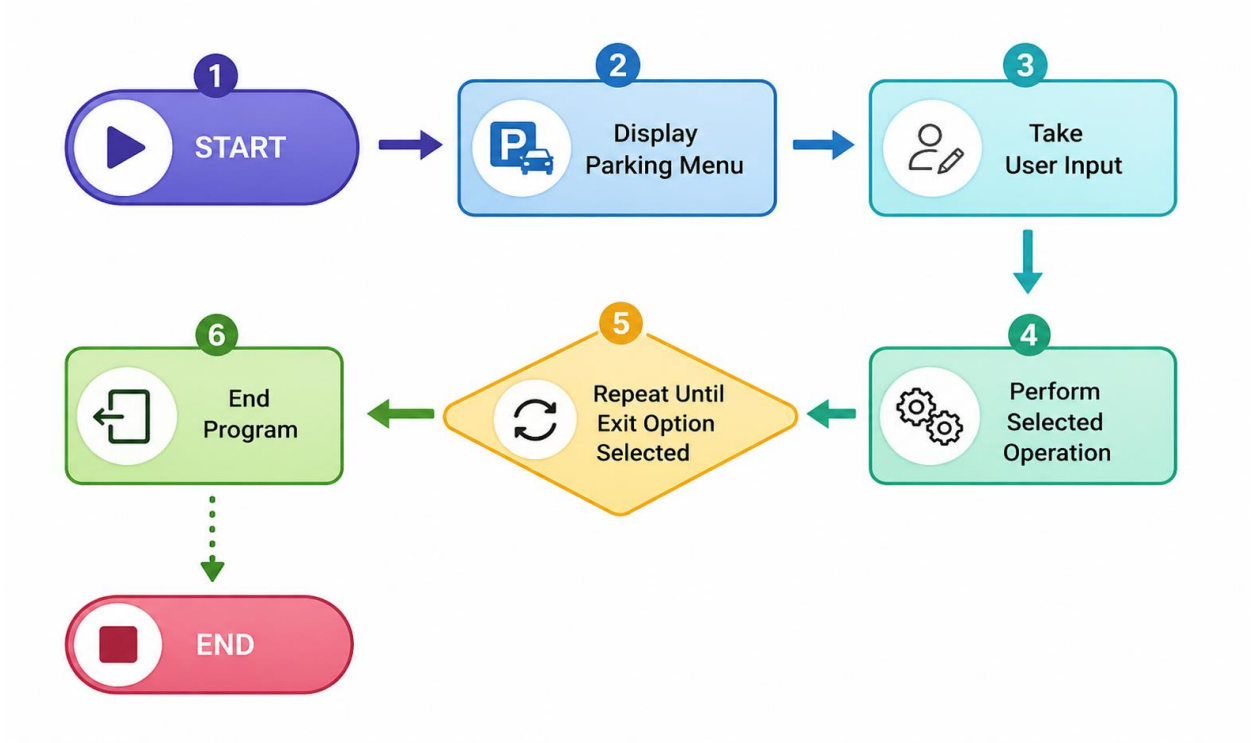
Features of Project

- Rikshaw parking entry
- Car parking entry
- Bus parking entry
- Record display system
- Delete last record using stack
- Parking full condition handling
- Wrong input handling
- Total amount calculation

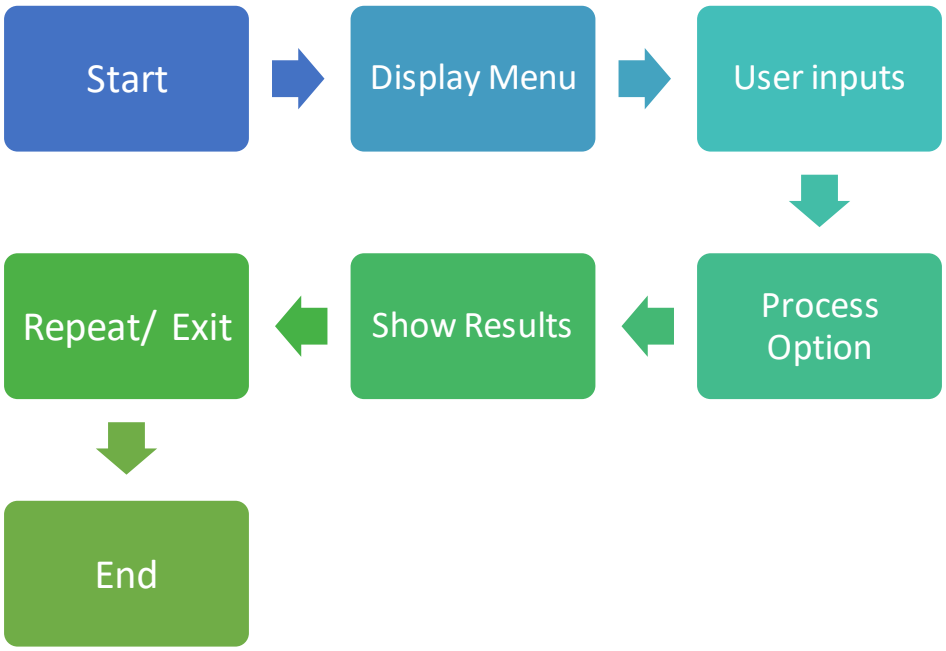
Software Requirements

Requirement	Description
Emulator	EMU8086
Operating Environment	DOS
Language	8086 Assembly
Processor	Intel 8086

Algorithm



Simple Flowchart



Complete Source Code with Comments

```
org 100h

jmp start

; ---------------- DATA ----------------

menu db '**************** MENU ****************',13,10,'$'
menu1 db 'Press 1 for rikshaw',13,10,'$'
menu2 db 'Press 2 for cars',13,10,'$'
menu3 db 'Press 3 for bus',13,10,'$'
menu4 db 'Press 4 to show record',13,10,'$'
menu5 db 'Press 5 to delete LAST record',13,10,'$'
menu6 db 'Press 6 to exit',13,10,'$'

msg1 db 13,10,'Parking Is Full','$'
msg2 db 13,10,'Wrong Input','$'
msg3 db 13,10,'No Record Found','$'

msg7 db 13,10,'Total Amount = ','$'
msg8 db 13,10,'Total Vehicles = ','$'
msg9 db 13,10,'Rikshaws = ','$'
msg10 db 13,10,'Cars = ','$'
msg11 db 13,10,'Buses = ','$'

msg12 db 13,10,'Last Record Deleted','$'

amount dw 0
count db 0

r db 0
c db 0
b db 0
```

```
stack db 0,0,0,0,0,0,0,0
top db 0

; ---------------- CODE ----------------
start:
main_loop:

mov dx,menu
mov ah,09h
int 21h

mov dx,menu1
mov ah,09h
int 21h

mov dx,menu2
mov ah,09h
int 21h

mov dx,menu3
mov ah,09h
int 21h

mov dx,menu4
mov ah,09h
int 21h
mov dx,menu5
mov ah,09h
int 21h

mov dx,menu6
mov ah,09h
int 21h
```

```
; -------- INPUT --------

mov ah,01h
int 21h
mov bl,al

cmp bl,'1'
jne check2
jmp rikshaw

check2:
cmp bl,'2'
jne check3
jmp car

check3:
cmp bl,'3'
jne check4
jmp bus

check4:
cmp bl,'4'
jne check5
jmp record

check5:
cmp bl,'5'
jne check6
jmp delete_record

check6:
cmp bl,'6'
jne wrong_input
jmp exit_program
```

```
; -------- WRONG INPUT --------

wrong_input:

mov dx,msg2
mov ah,09h
int 21h

jmp main_loop
```

```
; -------- RIKSHAW --------

rikshaw:

cmp byte [count],8
jae full

add word [amount],200

inc byte [count]
inc byte [r]

mov si,0
mov al,[top]
mov ah,0
mov si,ax

mov byte [stack+si],1

inc byte [top]

jmp main_loop
```

```
; -------- CAR --------

car:

cmp byte [count],8
jae full

add word [amount],300

inc byte [count]
inc byte [c]

mov si,0
mov al,[top]
mov ah,0
mov si,ax

mov byte [stack+si],2

inc byte [top]

jmp main_loop
```

```
; -------- BUS --------

bus:

cmp byte [count],8
jae full

add word [amount],400

inc byte [count]
inc byte [b]

mov si,0
mov al,[top]
mov ah,0
mov si,ax

mov byte [stack+si],3

inc byte [top]

jmp main_loop
```

```
; -------- FULL --------

full:

mov dx,msg1
mov ah,09h
int 21h

jmp main_loop
```

```
; -------- RECORD --------
record:
mov dx,msg7
mov ah,09h
int 21h
mov ax,[amount]
call print_num

call newline

mov dx,msg8
mov ah,09h
int 21h
mov al,[count]
call print_digit

call newline

mov dx,msg9
mov ah,09h
int 21h
mov al,[r]
call print_digit

call newline

mov dx,msg10
mov ah,09h
int 21h
mov al,[c]
call print_digit
```

```
call newline

jmp main_loop

; -------- DELETE LAST RECORD --------

delete_record:

cmp byte [top],0
je no_record

dec byte [top]

mov si,0
mov al,[top]
mov ah,0
mov si,ax

mov al,[stack+si]

cmp al,1
je remove_rikshaw

cmp al,2
je remove_car

cmp al,3
je remove_bus

jmp main_loop
```

```
; -------- REMOVE CAR --------

remove_car:

sub word [amount],300

dec byte [count]
dec byte [c]

jmp deleted

; -------- REMOVE BUS --------

remove_bus:

sub word [amount],400

dec byte [count]
dec byte [b]

jmp deleted
```

```
; -------- DELETED --------

deleted:

mov dx,msg12
mov ah,09h
int 21h

jmp main_loop

; -------- NO RECORD --------

no_record:

mov dx,msg3
mov ah,09h
int 21h

jmp main_loop

; -------- EXIT --------

exit_program:

mov ah,4Ch
int 21h
```

```
; -------- PRINT NUMBER --------

print_num:
push ax
push bx
push cx
push dx

cmp ax,0
jne convert

mov dl,'0'
mov ah,02h
int 21h
jmp done_print

convert:
mov bx,10
mov cx,0

divide:
mov dx,0
div bx

push dx
inc cx

cmp ax,0
jne divide

display:
pop dx
```

```
add dl,'0'
mov ah,02h
int 21h

loop display

done_print:

pop dx
pop cx
pop bx
pop ax

ret
```

```
; -------- NEWLINE --------

newline:

mov dl,13
mov ah,02h
int 21h

mov dl,10
mov ah,02h
int 21h

ret
```

Important Code Explanation

Menu Display

- mov dx,menu
- mov ah,09h
- int 21h

This code displays the menu string on the screen using DOS interrupt 21h.

User Input

- mov ah,01h
- int 21h

This interrupt takes a single character input from the keyboard.

Parking Amount Calculation

- add word [amount],300

This instruction adds parking charges to the total amount.

Stack Record Storage

- mov byte [stack+si],2

This stores vehicle type in stack memory.

Delete Last Record

- dec byte [top]

This removes the last inserted vehicle record from stack.

Variables Explanation

Variable	Purpose
amount	Stores total parking amount
count	Stores total vehicles
r	Stores rikshaw count
c	Stores car count
b	Stores bus count

stack	Stores vehicle records
top	Points to top of stack

Interrupts Used

INT 21h AH=01h

Used for keyboard input.

- mov ah,01h
- int 21h

INT 21h AH=02h

Used to display a single character.

- mov ah,02h
- int 21h

INT 21h AH=09h

Used to display string messages.

- mov ah,09h
- int 21h

INT 21h AH=4Ch

Used to terminate the program.

- mov ah,4Ch
- int 21h

Stack Working Explanation

The project uses a stack array to store vehicle records.

- Rikshaw = 1
- Car = 2
- Bus = 3

Whenever a vehicle is added:

- Value is pushed into stack
- top variable increases

When deleting:

- top decreases
- Last inserted record is removed

This follows the **LIFO (Last In First Out)** principle.

Output Screenshots Placeholders

Main Menu

```
Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c c:\assembly
Drive C is mounted as local directory c:\assembly\

Z:\>c:

C:\>ps.com
**************** MENU ****************
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
```

Vehicle Entry

```
1**************** MENU ****************
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
2**************** MENU ****************
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
3**************** MENU ****************
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
```

Record Display

```
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
4
Total Amount = 1800

Total Vehicles = 6

Rikshaws = 2

Cars = 2

Buses = 2
**************** MENU ****************
```

Delete Record

```
Last Record Deleted**************** MENU ****************
Press 1 for rikshaw
Press 2 for cars
Press 3 for bus
Press 4 to show record
Press 5 to delete LAST record
Press 6 to exit
4
Total Amount = 1400

Total Vehicles = 5

Rikshaws = 2

Cars = 2

Buses = 1
```

Advantages and Limitations

Advantages

- Fast execution
- Low memory usage
- Demonstrates low-level programming

- Uses stack implementation
- Simple and easy to understand

Limitations

- Limited parking size
- DOS-based interface
- No database support
- No graphical interface

Future Improvements

- Add graphical user interface
- Add database connectivity
- Increase parking capacity
- Add date and time system
- Add file handling for permanent storage

Conclusion

The Parking Management System successfully demonstrates the use of 8086 Assembly Language for real-world problem solving. The project manages parking records, calculates amounts, handles user inputs, and implements stack operations efficiently. It is a useful educational project for understanding Assembly Language programming concepts.